



can you propose a project plan to get to 90% coverage

The current overall coverage sits at 62% (18,405 statements, 6,966 missing). To reach 90%, you need to cover approximately 5,126 additional statements across ~66 files. Here is a structured sprint plan to get there.^[1]

Current State

- Total statements: 18,405^[1]
- Currently missing: 6,966 (need to reduce to ~1,840)^[1]
- Statements to cover: ~5,126

Sprint Plan Overview

The work breaks naturally into 4 sprints, targeting the areas with the largest uncovered statement counts first.

Sprint 1 — Reports Layer (Weeks 1–2)

Target: ~1,329 statements | 16 files

These are zero-coverage and near-zero files that are comparatively self-contained rendering modules — good early wins:^[1]

- `reports/mortgage/` — all 3 files are at 0% (158 stmts total); write integration tests using mock mortgage data
- `reports/property/property_integrator.py` — 0% (58 stmts);
`property_page_03_gauge_time_series.py` — 0% (111 stmts); `property_page_08_energy.py` — 0% (113 stmts)
- `reports/gauge/gauge_page_08_flood_history.py` — 13% (156 missing);
`gauge_page_11_current_risk.py` — 7% (127 missing); `gauge_integrator.py` — 7% (110 missing)
- `reports/property/property_generator.py` — 54% (109 missing); `risk_report_generator.py` — 61% (61 missing)

These report-page modules tend to have predictable input/output contracts, making parameterised tests straightforward.

Sprint 2 — Visualization Core & Layers (Weeks 3–4)

Target: ~1,080 statements | 10 files

The visual/ tree has near-zero coverage on its heaviest modules:^[1]

- `visual/core/visualizer.py` — 21% (167 missing); `data_loader.py` — 27% (141 missing); `map_builder.py` — 39% (69 missing)
- `visual/layer/mortgage_layer.py` — 15% (158 missing); `property_layer.py` — 16% (141 missing); `gauge_layer.py` — 18% (122 missing)
- `visual/utils/formatters.py` — 31% (89 missing); all 4 `data_extractors/` files are 10–14%

Strategy: use fixture-based tests with synthetic portfolio data (you already have generators in `port/rand/thames/`) and mock the `map/render` calls.

Sprint 3 — Port / Data Layer (Weeks 5–6)

Target: ~1,033 statements | 15 files

These are core data pipeline files where untested paths carry the highest risk:^[1]

- `port/cdm/gaugehd.py` — 0% (145 missing); `port/src/storm/stormts.py` — 16% (125 missing); `port/src/historical_eod.py` — 15% (123 missing)
- `port/rand/thames/stormts_random.py` — 11% (124 missing); `port/rand/thames/mortgage_random.py` — 68% (101 missing)
- `port/src/gauge/gaugehd/gaugehd.py` — 47% (47 missing); `gaugehd/nrfa.py` — 22% (46 missing)
- `port/src/property/propertyts.py` — 66% (70 missing); `port/src/mortgage.py` — 75% (42 missing)

The CDM (Common Data Model) files like `ctpy.py` and `gaugehd.py` should be tackled first as other layers depend on them.

Sprint 4 — Routes / API & Models (Weeks 7–8)

Target: $\sim 963 + 467 + 263 = \sim 1,693$ statements | remaining files

Routes are the final layer and easiest to test with Flask test client:^[1]

- `routes/governance/mrc_crud.py` — 18% (162 missing); `mrc_pdf.py` — 13% (86 missing); `mrc.py` — 24% (63 missing)
- `routes/propertyts/risk.py` — 7% (122 missing); `animation.py` — 8% (113 missing); `financial.py` — 11% (67 missing)
- `routes/prs.py` — 45% (98 missing); `routes/properties.py` — 43% (62 missing)

For models, the flood risk cluster is the priority:

- `models/floodrisk/risk_analytics.py` — 18%; `flood_risk_model.py` — 30%; `depth_damage.py` — 21%; `spatial.py` — 31%
- `models/hazard/pricing.py` — 13% (34 missing); `models/intensity/cli.py` — 10% (56 missing)
- `loaders/lookups.py` — 8% (143 missing) and `loaders/storm_loader.py` — 22% (65 missing)

Milestone Summary

Sprint	Area	Files	Stmts Covered	Projected Coverage
1	Reports	~16	~1,329	~69%
2	Visualization	~10	~1,080	~75%
3	Port / Data	~15	~1,033	~81%
4	Routes, Models, Loaders	~25	~1,684	90%+

Practical Notes

- **Quick wins first:** The 5 fully-zero files (`gaugehd.py`, `mortgage_generator.py`, `mortgage_page_01_title.py`, `property_integrator.py`, `property_page_03_gauge_time_series.py`) represent ~572 free statements — hit these in the first week.^[1]
- **Reuse synthetic generators:** Your `port/rand/thames/` random data generators are well-covered and ideal as test fixtures throughout.
- **Route tests via Flask test client:** Use `app.test_client()` for all `routes/` modules — these don't require complex mocking and can drive coverage quickly.
- **Add `# pragma: no cover`** to genuinely untestable paths (CLI entry points like `__main__.py`, OS-level file-open calls in `reports/utls/open_pdf.py`) to avoid inflating the missing count artificially.

✱

1. Coverage-report.pdf